



FASHION FUSION

IT Asset Management & Logistics System

Comprehensive Design Package

Document version 1.0 | Prepared June 2026
Owner: Group IT Department
Classification: Confidential — Internal Use

Document Control

Project	IT Asset Management & Logistics System (ITAMLS)
Organisation	Fashion Fusion (Retail Group — Head Office, Central Stock Room, Stores)
Document Type	End-to-end Design Package: architecture, data model, API, UX, workflows, roadmap
Version	1.0
Status	Draft for IT Steering Committee review
Audience	CIO/IT Director, IT Managers, Solution Architect, Dev Lead, Finance, Store Operations
Implementation Window	Indicative — 7 to 9 months from sign-off (see Roadmap)

Revision history

Version	Date	Author	Change
0.1	Jun 2026	IT Architecture	Initial scoping draft from brief
1.0	Jun 2026	IT Architecture	Full design package — architecture, DB, API, UX, roadmap

Table of Contents

1.	Executive Summary
2.	Business Context — Fashion Fusion
3.	Scope, Goals & Out-of-Scope
4.	System Architecture
5.	Technology Stack
6.	Data Model & Database Schema
7.	Entity Relationship Diagram (ERD)
8.	API Design
9.	Functional Modules (1–17)
10.	Permission Model & RBAC Matrix
11.	Key Workflows (BPMN-style)
12.	Fashion Fusion Store Standards Compliance
13.	UI Wireframes & Navigation
14.	Mobile Application Design
15.	Reporting & Dashboards
16.	Asset Lifecycle, Depreciation & Forecasting
17.	Integrations (LDAP/AD, Helpdesk, Email, Couriers)
18.	Security, Compliance & Audit Trail
19.	Non-Functional Requirements
20.	Implementation Roadmap
21.	Risks & Mitigations
22.	Future Scalability Recommendations
23.	Glossary
24.	Appendices

1. Executive Summary

The IT Asset Management & Logistics System (ITAMLS) is a single, authoritative platform for managing every IT asset, stock item, movement, repair, invoice and store-deployment event across Fashion Fusion. It replaces the patchwork of spreadsheets and ad-hoc trackers used by the Group IT function today, and gives the business — for the first time — a real-time, location-aware view of where every PC, monitor, router, switch, printer, UPS and accessory is, what it cost, who is using it, what it is worth, and when it must be replaced.

The platform is delivered as an ERP-style module purpose-built for the IT department, comprising seventeen functional modules grouped into four operational pillars: **(1) Inventory & Stock**, **(2) Logistics & Deployment**, **(3) Procurement & Finance**, and **(4) Service, Audit & Compliance**. A standout capability — Store Standards Compliance — compares every store against a defined IT equipment template, flagging missing or non-compliant items automatically. This single feature alone is expected to eliminate the bulk of manual audit effort and accelerate new-store openings by 30 to 50 percent.

The recommended stack is React, TypeScript and Tailwind on the front end; NestJS on the back end; PostgreSQL for transactional data; MinIO or any S3-compatible store for invoice, warranty and deployment documents; JWT plus Active Directory / LDAP for authentication; and Docker on Ubuntu for deployment. A React Native mobile companion app, designed offline-first, lets technicians scan asset QR codes, run store audits, receive and dispatch stock, capture photos, and view warranties from any tablet or phone.

Headline expected outcomes

100% visibility of every IT asset; ~70% reduction in audit time; sub-day store-deployment proposals; automated warranty alerts at 30/60/90 days; full depreciation and budget forecasting; and a measurable reduction in lost, untracked and duplicated equipment.

2. Business Context — Fashion Fusion

Fashion Fusion operates a head office, a central stock room and a growing portfolio of retail stores across multiple regions. The Group IT function is responsible for the procurement, deployment, support and decommissioning of every piece of IT equipment used in those stores — from point-of-sale PCs and slip printers to network switches, CCTV NVRs and UPS units. The current operating model is heavily manual and produces several recurring pain points:

- Stock visibility is fragmented across spreadsheets maintained by individual technicians.
- Store openings are slow because the equipment list, costs and dispatch plan are recreated each time.
- Warranty claims are missed because expiry dates are not tracked centrally.
- Repairs and RMAs are tracked by email, making it hard to know which assets are out of service.
- Invoices, delivery notes and quotations are stored across shared drives with no link to the asset.
- There is no single source of truth for what equipment every store *should* have.

Stakeholders

Stakeholder	Primary interest in the system
CIO / IT Director	Asset value, lifecycle KPIs, capex forecasting, compliance
IT Managers	Approvals, repair throughput, store compliance, team workload
Technicians	Mobile scanning, dispatch, audits, fault logging
Store Managers	Visibility of store assets, request equipment, sign for deliveries
Finance	Procurement workflow, invoice repository, depreciation, budget forecasts
Internal Audit	Audit trail, store standards compliance, variance reporting
Suppliers	(External) consume RFQs and POs via email/portal

3. Scope, Goals & Out-of-Scope

3.1 In-scope

- All IT hardware, accessories and consumables defined in the inventory taxonomy.
- Head office, central stock room and every retail store as tracked locations.
- Goods Received Voucher (GRV), Internal Branch Transfer (IBT) and Dispatch flows.
- Procurement requests, approvals, supplier and PO management.
- Repairs, RMAs, warranties and supplier-managed claims.
- Store deployment wizard with auto-generated equipment list and costed proposal.
- Asset auditing, barcode/QR scanning, mobile companion app, offline mode.
- Reporting and executive dashboards across asset value, lifecycle, repairs and procurement.
- Active Directory / LDAP authentication and role-based access control.

3.2 Out-of-scope (phase 1)

- Full helpdesk ticketing (the system *integrates* with the existing helpdesk rather than replacing it).
- POS system integration for sales transactions.
- Payroll, HR and general ledger postings (Finance receives exports rather than live posting).
- Customer-facing applications.

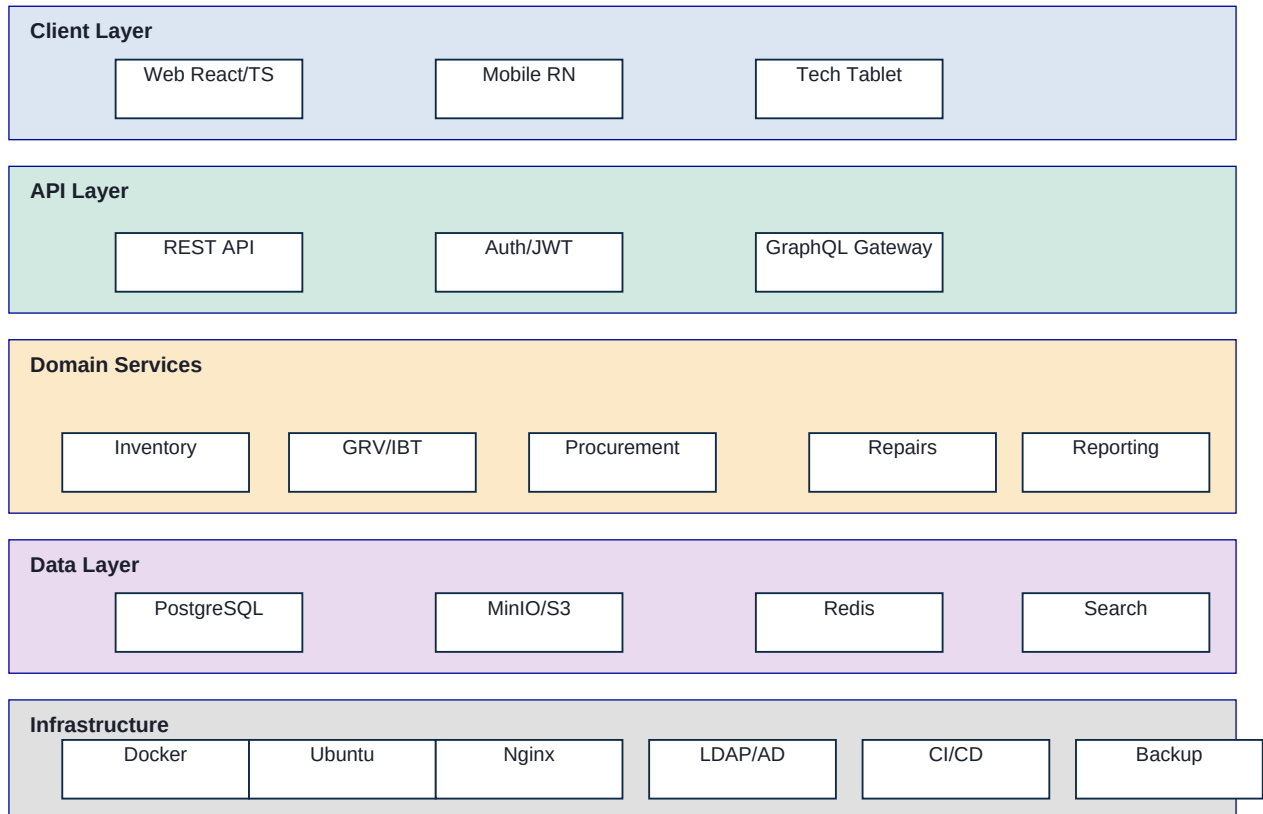
3.3 Success criteria

KPI	Target by end of Year 1
Asset record completeness	≥ 98% of assets carry serial, tag, location, warranty
Store standards compliance	≥ 95% of stores fully compliant against template
Audit cycle time per store	Reduced from 2 days to under 4 hours
New-store deployment proposal	Generated in under 1 working day (vs ~5 today)
Warranty claim recovery	≥ 90% of in-warranty repairs claimed back
Stock-room shrinkage	Reduced by ≥ 40% measured by year-end variance

4. System Architecture

ITAMLS is built as a containerised multi-tier application following a modular monolith pattern: a single deployable NestJS service is internally organised into bounded contexts (Inventory, Logistics, Procurement, Finance, Service, Identity, Reporting). This keeps operational complexity low for a small IT team while allowing any module to be carved out into its own service when scale requires it. PostgreSQL is the single transactional store; MinIO holds binary documents; Redis handles caching, rate limiting and background-job queuing.

4.1 Logical architecture



4.2 Bounded contexts

Context	Owns	Reads from
Identity	Users, roles, sessions, AD/LDAP sync	—
Catalog	Categories, subcategories, SKUs, templates	—
Inventory	Assets, stock, locations, movements, history	Catalog, Identity
Logistics	GRV, IBT, Dispatch, signatures	Inventory, Identity
Procurement	Suppliers, POs, requests, quotations	Inventory, Identity
Service	Repairs, RMAs, warranties, helpdesk links	Inventory, Procurement
Finance	Invoices, depreciation, costs, exports	Inventory, Procurement, Service
Reporting	Read-models, dashboards, exports	All

4.3 Deployment topology

Two Ubuntu LTS hosts behind an nginx reverse proxy run Docker Compose stacks for the API, the web front end, PostgreSQL (with streaming replication), MinIO (two nodes), and Redis. A separate backup target stores nightly Postgres base backups and continuous WAL plus MinIO bucket replication. The mobile app talks to the API over TLS. AD/LDAP runs on the existing corporate directory. Sizing for phase one: 4 vCPU / 16 GB RAM per host is comfortable for the expected load.

5. Technology Stack

Layer	Technology	Rationale
Web front end	React 18 + TypeScript + Vite + Tailwind	Required by brief; mature, fast iteration, strong component ecosystem.
UI library	shadcn/ui + Radix primitives	Accessible, themable, low lock-in.
State	TanStack Query + Zustand	Server-state caching plus minimal local state.
Mobile	React Native + Expo (managed) + WatermelonDB	Offline-first DB synced to API; QR scanning via expo-camera.
API	NestJS (Node.js 20) + TypeScript	Required by brief; modular structure, decorators, DI.
Auth	JWT (access + rotating refresh) + Passport-JWT	JWT for API tokens; AD/LDAP for user authentication and group sync.
DB	PostgreSQL 16	Required by brief; mature transactional store with rich JSONB.
File store	MinIO (S3 API)	Self-hostable; invoice, warranty and deployment-doc storage.
Cache / queue	Redis 7 + BullMQ	Background jobs: emails, OCR, depreciation runs, exports.
Search	PostgreSQL full-text + pg_trgm (Elastic optional)	Full-text search as a service in phase 1.
Reporting	Server-rendered PDF (puppeteer) + ExcelJS	PDF proposal, dispatch notes, audit reports; Excel exports.
Observability	OpenTelemetry → Loki + Grafana + Prometheus	Logs, metrics and traces in a single pane.
CI/CD	GitHub Actions or GitLab CI → Docker Registry	Reproducible builds; self-deploy.
Deployment	Docker on Ubuntu LTS behind nginx	Required by brief; low ops overhead.

6. Data Model & Database Schema

The schema is normalised, with a dedicated immutable history table that records every movement and status change of an asset. Soft-delete is used on master data; transactional records are append-only with status fields. All monetary values are stored in minor units (cents) as integers to avoid floating-point drift. Timestamps are stored in UTC.

6.1 Master data tables

Table	Key columns	Notes
users	id, email, ad_sid, role_id, store_id, is_active	Synced nightly from AD; role_id drives RBAC.
roles	id, name, description	Administrator, IT Manager, Technician, Store Manager, Finance, Auditor
permissions	id, code, description	Granular permissions referenced by role_permissions.
role_permissions	role_id, permission_id	Many-to-many.
categories	id, name, parent_id	Self-referencing for sub-categories.
suppliers	id, name, contact, tax_no, payment_terms	Used by GRV, repairs, procurement.
locations	id, code, type, name, region, parent_id	type ∈ {HO, STOCK_ROOM, STORE, REPAIR_BAY, IN_TRANSIT}
stores	id, location_id, code, region, template_id, opened_at, status	Location of type STORE.
store_templates	id, name (Small/Medium/Large/Flagship), description	Defines standard IT equipment composition.
store_template_items	template_id, category_id, required_qty	Drives Store Standards Compliance check.
skus	id, code, name, category_id, manufacturer, model, serialised, depreciation_years	Serialised for IT equipment, non-serialised for consumables/accessories.

6.2 Asset, stock and movement

Table	Key columns	Notes
assets	id, asset_tag, serial_no, sku_id, supplier_id, purchased_at, serialised	One-to-one serialised item. Asset tag is immutable. QR-coded ID.
asset_history	id, asset_id, event_type, from_location_id, to_location_id, referred_to_type_id, referred_to_id	Append-only table. event_type ∈ {PURCHASED, RECEIVED, DISPATCHED}
stock	id, sku_id, location_id, qty_on_hand, qty_reserved	Used for consumables (non-serialised) and for aggregated stock.
stock_movements	id, sku_id, from_location_id, to_location_id, qty_added, qty_removed	Append-only table for stock movements (or maintained via transfer).

6.3 Logistics tables

Table	Key columns	Notes
grv	id, supplier_id, invoice_no, invoice_date, delivered_at, lines_id, total_cents, received_by, status	GRV header, lines_id grv_lines
grv_lines	id, grv_id, sku_id, qty, unit_cost_cents, serial_no	Serialised SKUs expand into this table on confirmation.
ibt	id, code, from_location_id, to_location_id, status	Workflow: REQUESTED → APPROVED → DISPATCHED → BACKLOG
ibt_lines	id, ibt_id, asset_id (nullable), sku_id, qty	Asset_id present when transferring a serialised asset.
dispatches	id, ibt_id, store_id, technician_id, signed_by, signed_at	Stores the dispatch signature received for store acknowledgement.

6.4 Procurement, repairs & invoices

Table	Key columns	Notes
purchase_requests	id, code, requested_by, status, total_est_cents	Workflow: DRAFT → SUBMITTED → IT_APPROVED → FINANCE_APPROVED → ORDERED → CLOSED
purchase_request_lines	id, pr_id, sku_id, qty, est_unit_cost_cents	—
quotations	id, pr_id, supplier_id, file_url, total_cents, valid_until	—
purchase_orders	id, code, supplier_id, pr_id, status, total_cents, sent_at, expected_delivery	—
repairs	id, asset_id, status, fault_desc, supplier_id, sent_at, resolved_at, closed_at, at_supplier, returned	STATUS: LOGGED, SENT, AT_SUPPLIER, RETURNED, REPLACED, WRITTEN_OFF
invoices	id, doc_no, supplier_id, invoice_date, total_cents	Document, repository, linked to asset, linked to purchase order, linked to purchase request.

6.5 Service, audit & compliance

Table	Key columns	Notes
audits	id, store_id, scheduled_for, status, conducted_by, template_id	By template to store visit.
audit_lines	id, audit_id, asset_id, expected, found, condition	Missing/Damaged/Unrecorded flagged via variance.
store_compliance_view	(materialised view)	store_id, template_id, sku, required, installed, status.
alerts	id, type, severity, ref_table, ref_id, message, created_at	Warning, 30/60/90 day stock, missing equipment, repair delay, asset warranty, 30/60/90 day.
helpdesk_links	id, asset_id, ticket_external_id, system, opened_at, closed_at	Matched for tickets pulled from the existing helpdesk.
depreciation_runs	id, period, posted_at, total_depreciation_cents	Monthly straight-line job output.

6.6 Status enumerations

Entity	Allowed statuses
Asset	IN_STOCK, ALLOCATED, IN_STORE, IN_TRANSIT, REPAIR, DAMAGED, LOST, WRITTEN_OFF, DISPOSED
IBT	REQUESTED, APPROVED, DISPATCHED, IN_TRANSIT, RECEIVED, CANCELLED
Purchase Request	DRAFT, SUBMITTED, IT_APPROVED, FINANCE_APPROVED, ORDERED, CLOSED, REJECTED
Repair	LOGGED, SENT, AT_SUPPLIER, RETURNED, REPLACED, WRITTEN_OFF
Store	PLANNED, OPENING, OPEN, REMODEL, CLOSED
Audit	SCHEDULED, IN_PROGRESS, COMPLETED, OVERDUE

7. Entity Relationship Diagram

The ERD below shows the core entities and the high-level relationships between them. Field lists are illustrative — see Section 6 for the canonical schema.



Key relationships: every asset has many history rows; every store has one location and one store template; every IBT generates a dispatch on receipt; every GRV may link to an invoice and a purchase order; every repair links back to an asset and (optionally) a supplier invoice for the warranty claim.

8. API Design

The API is REST over HTTPS, JSON-encoded, versioned at `/api/v1`. All endpoints require a valid bearer JWT except `/auth/login` and health probes. Pagination is cursor-based (`?cursor=&limit=`). Errors use RFC 7807 problem details. A separate WebSocket channel pushes live updates for dispatch sign-offs, alerts and dashboards.

8.1 Resource map (selected)

Method	Path	Purpose
POST	<code>/auth/login</code>	Authenticate (AD/LDAP or local) → access + refresh JWT
POST	<code>/auth/refresh</code>	Rotate refresh token
GET	<code>/assets</code>	List/search assets; filters: status, location, category, warranty
POST	<code>/assets</code>	Create asset (typically via GRV)
GET	<code>/assets/{id}</code>	Asset detail + history + linked tickets + repairs
POST	<code>/assets/{id}/move</code>	Record a move (also writes <code>asset_history</code>)
GET	<code>/stock</code>	Stock by location; supports group-by
POST	<code>/grv</code>	Create GRV header
POST	<code>/grv/{id}/lines</code>	Append line, including serials
POST	<code>/grv/{id}/confirm</code>	Confirm GRV → creates asset rows + stock movements
POST	<code>/ibt</code>	Create transfer request
POST	<code>/ibt/{id}/approve</code>	IT Manager approval
POST	<code>/ibt/{id}/dispatch</code>	Dispatch + generate Dispatch Note PDF
POST	<code>/ibt/{id}/receive</code>	Store acknowledges receipt + e-signature
POST	<code>/dispatches/{id}/signature</code>	Upload signature blob (mobile)
GET	<code>/stores</code>	List stores
POST	<code>/stores/wizard</code>	Run store deployment wizard → cost + equipment list
GET	<code>/stores/{id}/compliance</code>	Compliance status vs template (■ ■ ■)
POST	<code>/purchase-requests</code>	Create PR
POST	<code>/purchase-requests/{id}/submit</code>	Submit for IT approval
POST	<code>/purchase-requests/{id}/approve</code>	IT/Finance approval step
POST	<code>/repairs</code>	Log fault
POST	<code>/repairs/{id}/transition</code>	Move to next status
GET	<code>/warranties/expiring?in=30 60 90</code>	Warranty expiry dashboard feed
POST	<code>/audits/{id}/scan</code>	Mobile scan endpoint; matches <code>asset_tag</code>
GET	<code>/reports/{name}</code>	Streamed PDF or XLSX export
GET	<code>/alerts</code>	Active alerts for current user

8.2 Conventions

- All write endpoints accept an Idempotency-Key header to make retries safe.
- All list endpoints support ?include= to expand related resources (e.g. include=history,invoices).
- All endpoints emit a structured audit-log event consumed by the Audit context.
- OpenAPI 3.1 spec is auto-generated from NestJS decorators and served at /api/v1/openapi.json.
- Webhooks (optional) for supplier portals: po.sent, grv.confirmed, asset.disposed.

9. Functional Modules

This section restates each of the seventeen modules from the brief with concrete behaviour, responsible roles, validation rules and integration points.

9.1 Inventory Management

Master register of every IT asset, accessory and consumable. Hardware is serialised and tracked individually; accessories and consumables are tracked by SKU and quantity.

Key behaviour

- Three category roots — Hardware, Accessories, Consumables — with sub-categories per the brief.
- Each serialised asset carries Asset Number (system-generated), Serial Number, Manufacturer, Model, Category, Supplier, Purchase Date, Cost, Current Value, Warranty Expiry, Condition and Status.
- Asset Number is unique, QR-printable and persistent across moves.
- Status transitions are guarded — e.g. an asset in REPAIR cannot be set to IN_STORE without passing through a Receive step.
- Bulk import via CSV for migration; row-level validation report on dry run.

Primary roles: Technician, IT Manager, Administrator

9.2 Multi-Location Stock Control

Real-time stock view by location: Head Office, Central Stock Room and every store.

Key behaviour

- Stock = serialised assets at a location + non-serialised quantities from stock_movements ledger.
- Drill-down: a store row expands into its full asset register (PCs, monitors, printers, network).
- Reorder levels per SKU per location feed the Low Stock alert and the Procurement dashboard.
- Region rollups for executive view (e.g. KZN, Gauteng, Western Cape).

Primary roles: Technician, Store Manager (own store), IT Manager

9.3 Goods Received Voucher (GRV)

Captures incoming stock against the supplier invoice and updates inventory atomically.

Key behaviour

- Header: supplier, invoice no, invoice date, delivery note, total. Lines: SKU, qty, unit cost, serials.
- Invoice copy uploaded to MinIO; OCR text indexed for full-text search.
- On confirm: assets are created (one per serial), stock_movements written, and warranty expiries calculated from purchase date + SKU warranty months.
- Three-way match against the related Purchase Order is highlighted in the UI.

Primary roles: Technician, IT Manager, Finance

9.4 Internal Branch Transfer (IBT)

Moves stock and assets between locations with a controlled, auditable workflow.

Key behaviour

- Workflow: REQUESTED → APPROVED → DISPATCHED → IN_TRANSIT → RECEIVED.
- Every state change is timestamped with the actor; receiving requires the store manager's signature.
- Courier name and tracking number are captured at dispatch and shown on the receiving screen.
- If items don't arrive within a configurable SLA, an alert is raised and the IBT is flagged as STUCK.

Primary roles: Technician, IT Manager, Store Manager

9.5 Dispatching

Generates a Dispatch Note PDF and captures an electronic signature on receipt.

Key behaviour

- Dispatch Note lists store, asset, serial, qty, technician and date — auto-numbered (DN-2026-00001).
- Mobile signature capture in the field; signature blob stored under the dispatch record.
- Successful receipt automatically transitions linked IBT to RECEIVED and updates asset locations.

Primary roles: Technician, Store Manager

9.6 Asset Tracking

End-to-end traceability of every major asset across its life.

Key behaviour

- Movement history pulled from asset_history (append-only) — never edited.
- Each asset page shows the full chain: Purchased → Stock Room → Store 010 → Repair → ...
- QR code on the asset opens the page directly when scanned.
- Currently assigned user, store and installation date are first-class fields.

Primary roles: Technician, IT Manager, Store Manager

9.7 Store Deployment Wizard

Generates an equipment list and a fully costed proposal for a new or remodelled store in minutes.

Key behaviour

- Inputs: store name, region, number of POS points, offices, printers, screens — plus a template choice (Small / Medium / Large / Flagship).
- Auto-generates required equipment list from the template, with overrides per input.
- Costs broken out: Equipment, Installation, Travel, Labour, Contingency — totalled.
- Outputs: a polished PDF Store Deployment Proposal that can be sent for approval.
- On approval, generates the corresponding Purchase Requests and reserves stock if available.

Primary roles: IT Manager, Finance, Technician

9.8 Procurement

Structured request → approval → order → delivery chain with a supplier database.

Key behaviour

- Workflow: Technician → IT Manager → Finance → Approved → PO → Delivery.
- Quotations are uploaded against the PR; the chosen quotation seeds the PO.
- Supplier database holds contact, payment terms, lead times and historical performance score.
- Delivery status pulls from the linked GRV.

Primary roles: Technician, IT Manager, Finance

9.9 Invoice Repository

Single searchable archive of supplier invoices, quotations, delivery notes and warranty docs.

Key behaviour

- All documents stored on MinIO; OCR run asynchronously via Redis-queued worker.
- Search by supplier, asset, invoice number or free text against OCR'd content.
- Bidirectional linking — invoice ↔ asset(s), invoice ↔ GRV, invoice ↔ PO, invoice ↔ repair.

Primary roles: Finance, IT Manager, Auditor

9.10 Repairs & RMA

Tracks every fault from log through resolution, including warranty claims.

Key behaviour

- Statuses: Fault Logged → Sent For Repair → Supplier Repair → Returned / Replaced / Written Off.
- Cost, supplier, warranty-claim flag and outcome captured on each repair.
- Repeat-offender report surfaces assets with ≥ 3 repairs in 12 months for replacement planning.

Primary roles: Technician, IT Manager

9.11 Warranty Management

Automatic warranty tracking with rolling expiry alerts.

Key behaviour

- Warranty expiry computed from purchase date + warranty months on the SKU; overridable per asset.
- Dashboard tiles: expiring in 30 / 60 / 90 days.
- Email + in-app notifications generated by a nightly job; suppressed once an asset is RETIRED.

Primary roles: IT Manager, Finance

9.12 Asset Auditing

Scheduled store audits with mobile barcode scanning and variance reporting.

Key behaviour

- Audit sheet is auto-generated from the store's expected asset register.
- Technician scans each asset; mismatches are flagged Missing / Damaged / Unrecorded.
- Photos can be attached to variances; audit report exported as PDF and stored against the store.

Primary roles: Technician, IT Manager, Auditor

9.13 Barcode / QR System

Every asset receives a printable QR code that resolves to its detail page.

Key behaviour

- QR codes generated server-side as PNG/SVG and printable on label-printer stock.
- Mobile app, tablet or phone camera scans the QR for instant asset lookup.
- Same QR is embedded on dispatch notes for fast receive-and-confirm.

Primary roles: Technician, Store Manager

9.14 Reporting Module

Executive dashboard plus operational reports across asset, inventory, procurement and repairs.

Key behaviour

- Executive: asset count, value, depreciation, by category/store/region.
- Inventory: stock levels, low stock, out of stock, slow-moving.
- Procurement: by supplier, by month, by category.
- Repairs: cost, failure trends, top problem assets.
- Store: assets per store, equipment value, audit results.
- All reports exportable to PDF and Excel.

Primary roles: All roles (filtered by RBAC)

9.15 IT Helpdesk Integration

Two-way link between assets and helpdesk tickets so fault history is complete.

Key behaviour

- Tickets are mirrored into helpdesk_links by a polling adapter (Freshservice / Jira SM / Spiceworks).
- Each ticket auto-associates with an asset by tag, store + category, or technician selection.
- Asset page shows open and historical tickets alongside repairs.

Primary roles: Technician, IT Manager

9.16 User Roles

Strict role-based access control underpinned by AD/LDAP group membership.

Key behaviour

- Six default roles: Administrator, IT Manager, Technician, Store Manager, Finance, Auditor.
- Scope filters: store managers see only their store; auditors see read-only across all.
- All admin actions are written to an immutable audit log.

Primary roles: Administrator

9.17 Mobile Application

React Native companion app for technicians and store managers, offline-first.

Key behaviour

- Scan assets and look them up offline (cached register synced nightly + on demand).
- Run store audits without connectivity; queue results to sync on reconnect.
- Receive stock at a store, dispatch from stock room, capture signatures and photos.
- View warranty status and open faults from the asset page.

Primary roles: Technician, Store Manager

10. Permission Model & RBAC Matrix

Permissions are coded as resource:action. Roles are bundles of permissions. AD/LDAP groups are mapped onto roles at login. The matrix below summarises the default mapping (✓ = full, R = read-only, S = scoped to own store, blank = none).

Capability	Admin	IT Mgr	Tech	Store Mgr	Finance	Auditor
Manage users and roles	✓					
Create / edit assets	✓	✓	✓			
View assets	✓	✓	✓	S	R	R
Receive stock (GRV)	✓	✓	✓		R	R
Approve IBT	✓	✓				
Dispatch stock	✓	✓	✓			
Acknowledge dispatch (signature)	✓	✓	✓	S		
Run store deployment wizard	✓	✓			R	R
Submit purchase request	✓	✓	✓	S		
Approve PR (IT step)	✓	✓				
Approve PR (Finance step)	✓				✓	
Manage suppliers	✓	✓			✓	R
Log / progress repairs	✓	✓	✓	S		
View warranty dashboard	✓	✓	✓	S	✓	R
Run store audit	✓	✓	✓			R
View executive dashboard	✓	✓	R		✓	R
Export reports (PDF/XLSX)	✓	✓	R	S	✓	✓
View audit log	✓	R			R	✓

11. Key Workflows

11.1 Internal Branch Transfer (IBT)

IBT lifecycle



Initiated by a technician or store manager; approved by the IT Manager; dispatched from the source location with a generated Dispatch Note; tracked through transit; and confirmed at the destination with an electronic signature. Asset locations and statuses update automatically at each step.

11.2 Procurement

Procurement lifecycle



Two-step approval ensures both the IT need and the finance commitment are signed off before any supplier order leaves the building. Delivered items are matched back to the PR via the GRV.

11.3 Repair / RMA

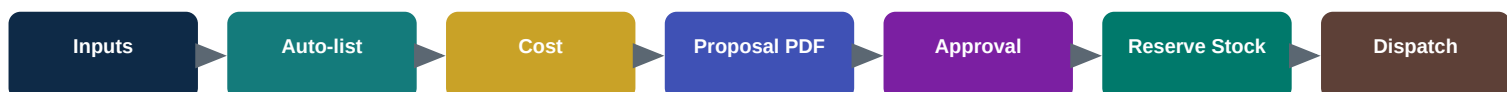
Repair lifecycle



Faults can be raised from a helpdesk ticket, mobile scan, or directly on the asset page. Warranty claims are flagged at log time and tracked separately for financial recovery.

11.4 Store Deployment

New store deployment



The wizard takes a small set of inputs, expands them against the chosen store template, costs them, and produces a polished PDF proposal. On approval the system creates the Purchase Requests for any missing items and reserves available stock for the new store.

11.5 Asset Lifecycle



Lifecycle stage is computed from asset age and repair history. Replacement-Due triggers a suggested PR; Retired triggers a disposal record and asset write-off entry.

12. Fashion Fusion Store Standards Compliance

Store Standards Compliance is the system's source of truth for what every Fashion Fusion store must contain. Four templates are pre-loaded; new templates can be created at any time. Each template lists categories and required quantities; the compliance engine compares the installed register against the template and flags every gap.

12.1 Default templates

Item	Small Store	Medium Store	Large Store	Flagship
POS PCs	2	4	6	8
POS Monitors	2	4	6	8
Slip Printers	2	4	6	8
Barcode Scanners	2	4	6	8
Office PCs	1	1	2	3
Office Monitors	1	1	2	3
Label Printers	1	1	2	2
Routers	1	1	1	1
Switches	1	1	2	2
Access Points	1	2	3	4
UPS Units	1	1	2	2
CCTV NVR	1	1	1	1

12.2 Compliance indicator

On every store page the system shows a compliance panel with three states per line:

State	Meaning	Trigger
■ Installed / In-Store	Units installed and present and operational	qty_installed ≥ required_qty and status ∈ {IN_STORE}
■ Partially Degraded / In-Store	Units in REPAIR / DAMAGED	qty_installed < required_qty OR any unit not IN_STORE
■ Not Installed / In-Store	Units still required for category at the store	qty_installed = 0

The same data drives the rollout at region and group level — answering questions like 'how many Large stores are missing a backup UPS?' in a single click. It also feeds the audit pre-flight check, so a technician arriving on site already knows what to expect.

13. UI Wireframes & Navigation

13.1 Information architecture

The web app uses a persistent left rail with role-aware sections. Each top-level item collapses to a secondary panel for sub-pages, keeping deep flows accessible in two clicks.

Top-level	Sub-pages
Dashboard	Executive · Operations · My Tasks
Inventory	Assets · Stock by Location · Categories · SKUs · Import
Logistics	GRV · IBT · Dispatches · Couriers
Procurement	Requests · Quotations · POs · Suppliers
Service	Repairs · Warranties · Helpdesk Tickets
Stores	Store List · Store Deployment Wizard · Compliance · Audits · Templates
Reports	Asset · Inventory · Procurement · Repairs · Store · Custom
Admin	Users · Roles · Permissions · Audit Log · Settings · Integrations

13.2 Key screens (described wireframes)

Asset Detail

Header strip with asset tag, QR thumbnail, status pill, current location and assigned user. Three tabs: **Overview** (specs, cost, value, warranty, supplier, invoice link), **History** (vertical timeline of movements with actor and date), and **Service** (open repairs, helpdesk tickets, attachments). A right-rail action menu offers Move, Reassign, Log Fault, Mark Disposed, Print Label.

Store Compliance

Left: store summary card (region, template, opened date, technician). Centre: compliance grid with a row per required category, columns Required / Installed / Status with the ■■■ chips from Section 12. Right: actionable side panel — Raise PR for missing items, Schedule Audit, View on Map. A small sparkline shows compliance trend over the last 12 months.

Store Deployment Wizard

Five-step horizontal stepper: **Basics** (name, region, opening date), **Layout** (POS, offices, printers, screens), **Template** (Small/Medium/Large/Flagship), **Costs** (line-itemed equipment plus installation, travel, labour, contingency — all editable), **Review & Generate** (creates PDF proposal and draft Purchase Requests). Each step shows a running total in the side panel.

Dispatch Note & Receive Screen

Dispatch Note PDF: header with store address, dispatch number, date and technician; line table with category, model, serial, qty; signature box and QR code that resolves to the live receive screen. Receive screen: scan each line to confirm, capture signature, optional photo, submit — everything works fully offline on the mobile app and syncs when reconnected.

Executive Dashboard

Six KPI tiles across the top — Total Assets, Asset Value, Depreciated Value, Stores Compliant, Open Repairs, Warranties Expiring (90 d). Below, four charts in a 2x2 grid — Assets by Category, Assets by Region, Procurement Spend by Month, Repair Cost by Supplier. A right-rail Alerts panel lists the top items requiring attention with one-click drilldowns.

14. Mobile Application Design

The mobile app is the everyday tool for technicians and store managers. It is built once with React Native (Expo managed) and ships to both iOS and Android. WatermelonDB provides a reactive local store that is synchronised to the API; this is what makes the app robust in stores with patchy connectivity.

14.1 Core flows

- Login (AD/LDAP) — credentials cached in the OS keychain.
- Scan asset — opens the asset detail card; supports torch and auto-focus.
- Audit store — pull the audit sheet, scan each asset, mark variances and capture photos.
- Receive dispatch — scan dispatched lines, sign on screen, submit.
- Dispatch from stock room — pick lines, capture courier & tracking, sign out.
- View warranties for the store — sortable by expiry date.
- Log fault — photo + free text + asset link.

14.2 Offline behaviour

- Every screen is hydrated from the local DB; network is opportunistic.
- Mutations are queued with idempotency keys and replayed on reconnect.
- Conflicting server changes surface as a soft merge with a clear UI prompt.
- Critical reference data (asset register for the assigned store, store template, suppliers) is pre-synced on login and refreshed nightly.

14.3 Wireframe summary

Bottom tab bar: **Home** (my tasks), **Scan** (full-screen camera, persistent floating scan button on every screen), **Audit**, **Stock**, **More**. Top app bar shows the current location context and a sync indicator with a queued-mutations counter.

15. Reporting & Dashboards

Report	Audience	Key fields	Format
Asset Summary by Category	Exec, IT Mgr	Category, count, value, depreciated value	PDF + XLSX
Assets by Store / Region	Exec, IT Mgr	Region, store, count, value	PDF + XLSX
Asset Value & Depreciation	Finance, Exec	Purchase cost, book value, accum. depreciation	PDF + XLSX
Stock Levels	IT Mgr, Tech	Location, SKU, on hand, reorder	XLSX
Low / Out of Stock	IT Mgr	SKU, location, on hand, days below reorder	PDF + XLSX
Slow-moving Stock	Finance, IT Mgr	SKU, last movement, value	XLSX
Purchases by Supplier	Finance	Supplier, count, spend, on-time %	PDF + XLSX
Purchases by Month / Category	Finance, Exec	Period, category, spend	PDF + XLSX
Repair Costs	IT Mgr, Finance	Asset, supplier, cost, in/out of warranty	PDF + XLSX
Failure Trends	IT Mgr	Category, model, count, MTBF estimate	PDF + XLSX
Top Problem Assets	IT Mgr	Asset, repair count, total cost	PDF + XLSX
Assets per Store	Store Mgr, IT Mgr	Store, category, count	PDF
Store Equipment Value	Finance, Exec	Store, category, value	XLSX
Store Audit Results	IT Mgr, Auditor	Store, audit date, variances	PDF
Warranty Expiry (30/60/90)	IT Mgr, Finance	Asset, supplier, expires in	PDF + XLSX

16. Asset Lifecycle, Depreciation & Forecasting

16.1 Depreciation

The system uses straight-line depreciation by default, with the useful life set on each SKU (typical defaults: PCs 4 years, monitors 5, printers 4, network gear 5, UPS 3, mobile phones 2). A monthly job posts the period's depreciation; current value is purchase cost minus accumulated depreciation, floored at the residual. Reducing-balance can be enabled per category for assets where the business prefers it.

16.2 Budget forecasting

By combining asset age, lifecycle stage and replacement cost, the system projects the next 12 months of capital replacement spend, by month and category. Forecasts are recomputed nightly and presented as a stacked bar on the Finance dashboard with drill-down to the underlying assets.

16.3 Lifecycle stages



Stages are derived: **New** < 3 months, **Active** within useful life, **Aging** ≥ 75% of useful life or ≥ 2 repairs, **Replacement Due** at end of useful life or ≥ 3 repairs in 12 months, **Retired** when status ∈ {WRITTEN_OFF, DISPOSED}.

17. Integrations

System	Direction	Mechanism	Phase
Active Directory / LDAP	Inbound (auth + group sync)	Passport-LDAP + nightly sync job	1
Email (SMTP relay)	Outbound	Notification service (BullMQ queue)	1
Helpdesk (Freshservice / Jira SM / Spiceworks)	Inbound (tickets)	REST adapter; webhook in + polling out	1
Courier (Aramex / Fastway / Dawn Wing)	Inbound (tracking)	Polling adapter for tracking number	2
Finance ERP (Sage / SAP / Pastel)	Outbound (depreciation, costs)	Scheduled CSV/JSON export to SFTP	2
SMS gateway	Outbound (delivery confirmations)	Provider REST API	2
Microsoft Teams / Slack	Outbound (alerts)	Incoming webhook	2
Supplier portal (RFQ)	Outbound	Signed email + PDF + tracker link	3

18. Security, Compliance & Audit Trail

- Transport: TLS 1.2+ everywhere; HSTS enabled on the front end.
- Auth: JWT (short-lived access + rotating refresh) bound to a session row that can be revoked.
- Identity: primary login via AD/LDAP; local accounts only for break-glass admin.
- Authorisation: RBAC with scoped permissions; store managers limited to their own store.
- Secrets: stored in environment variables sourced from a vault (HashiCorp Vault / Docker secrets).
- Data at rest: PostgreSQL with disk encryption; MinIO server-side encryption on the document bucket.
- Backups: PostgreSQL base + WAL nightly with 30-day retention; MinIO bucket replication.
- Audit log: append-only audit_events table records actor, action, target, payload-hash, timestamp.
- Privacy: assigned-user data is limited to name and AD ID; no personal IDs stored.
- Penetration testing: external assessment before go-live; OWASP ASVS L2 as the target baseline.

19. Non-Functional Requirements

Attribute	Target
Availability	99.5% in business hours (08:00–20:00 local), 99.0% overall
RPO / RTO	RPO ≤ 15 minutes (WAL archive); RTO ≤ 2 hours
API latency	p95 ≤ 300 ms for read; p95 ≤ 800 ms for write
Concurrent users	200 web + 100 mobile sustained; 500 burst
Data volume (year 3)	≤ 5 GB transactional; ≤ 200 GB documents
Browser support	Latest 2 versions of Chrome, Edge, Firefox, Safari
Mobile OS support	iOS 15+ and Android 10+
Accessibility	WCAG 2.1 AA on the web app
Localisation	English baseline; ZAR currency default; locale-aware date/number
Audit retention	7 years for financial-relevant events

20. Implementation Roadmap

Delivery is planned in four phases over roughly seven to nine months. Each phase ends with a production-quality release and a go/no-go review.

Phase	Duration	Headline outcome	Includes
0 — Discovery & Setup	3 weeks	Confirmed design, environments	Sign-off of this document, repo & CI/CD, infra provisioning, AD/LDAP
1 — Core Inventory & Logistics	10 weeks	Single source of truth for assets	Auth + RBAC, Identity, Catalog, Inventory (Modules 1, 2, 6), GRV (3),
2 — Procurement, Repairs & Mobile	8 weeks	End-to-end procurement and field operations	Procurement (8), Repairs & RMA (10), Helpdesk Integration (15)
3 — Stores, Compliance & Reporting	8 weeks	Compliance and exec reporting	Store Deployment Wizard (7), Auditing (12), Reporting (14), Store Sta
4 — Hardening & Cutover	4 weeks	Production cutover and training	Pen test, load test, UAT, data migration from spreadsheets, train-the-t

Indicative team

- 1 Tech Lead / Architect (full time)
- 2 Full-stack engineers (NestJS + React)
- 1 Mobile engineer (React Native)
- 0.5 UX designer
- 0.5 QA engineer
- 0.25 DevOps engineer
- Product Owner from Group IT (full time)

21. Risks & Mitigations

Risk	Likelihood	Impact	Mitigation
Migration data quality from spreadsheets is poor	High	High	Dry-run import with row-level validation; data-cleansing sprint in Phase 0; pilot with one store
Store connectivity in remote locations	Medium	Medium	Mobile app is offline-first; explicit sync indicator; nightly bulk sync from the cloud
AD/LDAP downtime blocks login	Low	High	Cached credential validation for 8 hours; local break-glass admin; documented escalation
User adoption (especially at stores)	Medium	High	Store-manager-centred UX; on-site training; champions per region; usage incentives
Scope creep into helpdesk replacement	Medium	Medium	Explicit integration approach; phase-gated change control.
Vendor lock-in on document store	Low	Low	MinIO speaks the S3 API — interchangeable with AWS S3 or Backblaze B2
Performance at year 3 data volumes	Low	Medium	Partition asset_history by year; add read replicas; pre-aggregated dashboards

22. Future Scalability Recommendations

- **Multi-tenant / multi-brand.** Add `tenant_id` to all tables and a tenant-aware middleware so the same platform can serve sister brands (e.g. a future apparel sub-brand) without code forks.
- **Service extraction.** Reporting and Service contexts are natural candidates for separate services once read traffic justifies it; the existing module boundaries make this low-risk.
- **Event streaming.** Replace the in-process domain-event bus with Kafka or NATS when downstream consumers (BI, supplier portal) appear.
- **Data lake feed.** CDC from PostgreSQL via Debezium into a warehouse (BigQuery / Snowflake / DuckDB) for self-service analytics.
- **AI features.** (i) Auto-classification of OCR'd invoices to categories; (ii) anomaly detection on repair patterns; (iii) demand forecasting for consumables; (iv) chat-style asset lookup.
- **Native edge.** A per-store mini cache (Pi or NUC) running a read-replica for fully resilient offline operation in dense store estates.
- **Compliance.** If the group lists publicly or trades internationally, layer ISO 27001 / SOC 2 controls on top of the existing audit log and RBAC.
- **Supplier portal.** A scoped external portal where suppliers acknowledge POs, upload invoices and lodge advance shipping notices — closing the loop on procurement.

23. Glossary

Term	Definition
Asset Tag	System-generated identifier printed as a QR code and physically affixed to the asset.
GRV	Goods Received Voucher — record of stock received from a supplier.
IBT	Internal Branch Transfer — movement of stock or an asset between locations.
RMA	Return Merchandise Authorisation — supplier-side repair or replacement process.
SKU	Stock Keeping Unit — the catalogue identifier for a model/variant.
WAL	Write-ahead log — PostgreSQL's mechanism for point-in-time recovery.
MTBF	Mean Time Between Failures — used in Failure Trend reports.
RBAC	Role-Based Access Control.
Template	Defined IT equipment composition for a store size (Small/Medium/Large/Flagship).
Compliance	The match between a store's installed register and its template.

24. Appendices

Appendix A — Category taxonomy

Root	Sub-categories
Hardware	Desktop PCs, Laptops, POS PCs, Servers, Routers, Switches, Access Points, Printers (incl. Slip and Label), Barcode S
Accessories	Keyboards, Mice, HDMI / VGA / DisplayPort / Power / Network / USB Cables, Power Adaptors
Consumables	Printer Toners, Printer Ink, Labels, Receipt Rolls

Appendix B — Sample Dispatch Note line format

Field	Example
Dispatch No.	DN-2026-00187
From	Central Stock Room — Durban
To	Store 012 — Gateway
Technician	T. Naidoo
Asset	HP ProDesk 405 G9 SFF — POS PC
Serial No.	CN12345ABC
Quantity	1
Date dispatched	14 June 2026
Courier / Tracking	Aramex / 12345678ZA
Signature	<i>captured on receipt</i>

Appendix C — Sample executive KPI tile contract

GET /api/v1/dashboards/executive → { total_assets:int, asset_value_cents:int, depreciated_value_cents:int, stores_compliant_pct:number, open_repairs:int, warranties_expiring_90d:int }

Appendix D — Sign-off

Role	Name	Signature	Date
CIO / IT Director			
Head of Finance			
Head of Store Operations			
Project Sponsor			